CS6310 – Software Architecture & Design
Assignment #3 [100 points]: Pokémon Tournament Project – Phase 3 (v0)
"Welcome to the Poke-Dome!"
Spring 2025 Semester – created by the CS6310 Instructional Team

Updates to the Core Functionality

The following updates to the system must be performed by every team, as opposed to the varied and assigned and selected Functional and Non-Functional Additions that vary for each team. The updates include: [1] expanding the Pokémon battles into tournaments; [2] implementing a Graphical User Interface (GUI); and [3] improving the error checking and handling capabilities of your system.

Disclaimer

This scenario has been developed solely for this course. Any similarities or differences between this scenario and any of the programs at Georgia Tech programs are purely coincidental.

[1] Goals for this Project Phase

Your goal for this phase of the project is to build on the individual Pokémon battles to support Pokémon tournaments. More specifically, you must design, develop, and implement the following commands during this phase in addition to your functional and non-functional requirements:

- **`displayinfo,<Pokemon>`**
- **`tournament,<Pokemon1>,<Pokemon2>,...,<PokemonN>`**
- **`battle,<Pokemon_1>,<Pokemon_2>`**
- **`// comments`**
- **`setseed,<Value>`**
- **`removeseed`**
- **`stop`**

A key aspect of your system is that new Pokémon will be introduced to your system as objects for testing purposes after submission. Your system will need to interact with the new Pokémon objects dynamically. Each Pokémon object must manage certain aspects of its own state and behavior – for example, each Pokémon must:

- Be able to initialize itself as needed and prepare itself for battle
- Manage its own health/hit points
- Be aware of its attack skills and its defensive skills
- Identify which attack and/or defense it will use for a given round while battling another Pokémon
- Determine how the damage levied by an attack against it will affect its health – for example, will a prior defensive action minimize the damage

You should not "hardcode" these effects into your system – rather, your system must let the Pokémon manage those aspects themselves. Also, even though we've shown you examples of "classic" Pokémon that have already been created, be aware that we will introduce new Pokémon when testing your system that will have different health levels and types of attacking and defensive skills.

Given the need to handle dynamic, run-time classes and objects, we strongly recommend that you investigate Java Reflection, and consider it as an approach (or part of your approach) to developing your system. Here are some sites that might be helpful:
https://www.oracle.com/technical-resources/articles/java/javareflection.html
https://docs.oracle.com/javase/tutorial/reflect/
https://download.java.net/java/early_access/panama/docs/api/java.base/java/lang/reflect/package-summary.html

A Pokémon determines what it will do – attack or defend – based on the ratio of its current hit points divided by the maximum number of hit points:

- If the ratio is greater than or equal to 0.7, then the Pokémon is still relatively strong, and will be very aggressive. In this case, there is an 80% likelihood that the Pokémon will decide to attack.
- If the ratio is less than 0.7, but still greater than or equal to 0.3, then the Pokémon is noticeably weaker than the start of the battle and will act in a more balanced manner. In this case, there is a 50% likelihood that the Pokémon will decide to attack.
- Finally, if the ratio is less than 0.3, then the Pokémon is significantly weaker and will act much more cautiously. In this case, there is only a 30% likelihood that the Pokémon will decide to attack.

In all these cases, if the Pokémon does not decide to attack, then it will defend itself. When the Pokémon attacks, then your system must select one of the Pokémon's attack skills randomly with an even likelihood for all the possible skills (i.e., a uniform probability distribution). Similarly, if the Pokémon decides to defend itself instead of attacking, then your system must select one of the Pokémon's defensive skills randomly with an even likelihood for all the possible skills.

Attacking skills and defensive skills are all identified by a name and a damage level. When a Pokémon launches an attack, your system must consider the level of damage that might be inflicted on the Pokémon being attacked. However, if the Pokémon being attacked selected a defensive skill on its prior action, then the full damage from the attacking skill might be limited – or even eliminated – by the defensive skill.

- If there was a defensive action on the prior turn, then the limited damage (i.e., full attack damage level – defensive damage level) is assessed.
- If there was no defensive action on the prior turn, then the full attack damage level is assessed.

After the attacking and defensive actions have been assessed, and the attacked Pokémon's hit points have been updated appropriately, your system must confirm that the Pokémon still has hit points. Finally, all fractional damage values must be rounded up to the next integer value (i.e., ceiling).

Randomness and Simulation Consistency

There are a number of moments during the **`battle()`** or **`tournament()`** processing that require decisions to be made based on a random value. Your system must use Java's Random capabilities to manage and generate all random values that are needed. To support a pseudo-random but consistent sequence of values, the Random library allows you to use a seed value to initialize the

value generation process. This supports the unpredictable behavior of Pokémon (to make things fun & interesting) while also allowing us to identify errors and other unexpected actions consistently.

Generating and Using Random Values in a Specific Manner

- Controlling the Pokémon's Actions: Each Pokémon will make several decisions during a battle. Consequently, each Pokémon must maintain a special "seed" value (as an integer) that must be used – once provided by the user/test case – to initialize its random value sequences. And if the **`setseed,<Value>`** command is used, then **`<Pokemon_1>`** must use **`<Value>`** as its random seed while **`<Pokemon_2>`** must use **`<Value + 1>`** as its random seed. This ensures that the Pokémon do not completely "mirror" (i.e., duplicate) each other's actions, which makes the battles much more interesting. Since a tournament is a sequence of battles, you will reuse **`<Value> & <Value + 1>`** for all of the battles that the tournament needs.

- Determining if a Pokémon will Attack or Defend: A Pokémon will vary its likelihood to attack versus defending itself (e.g., "aggressiveness") based on its current hit points. This likelihood (i.e., decision) to attack is then given as a probability, and your system must generate a random integer value between 0 to 9 (inclusively). It must attack if – and only if – the value is greater than or equal to the appropriate likelihood (i.e., "aggressiveness" percentage).

  Suppose that the Pokémon has a 60% likelihood of attacking. This would also mean there is a 40% likelihood of defending. So our threshold becomes the 4th number that random could generate. When asking the question did they meet the threshold or not, the "yes" answer is the top part of the range & the "no" answer is the bottom part of the range. In the case of [0-9] this means the value to check would be 3. So [0-3] becomes the defensive range or the "no you do not attack" & [4-9] becomes the attack range or the "yes you do attack". If the random value generated is between 4 and 9 (inclusive), then the Pokémon must attack; otherwise, if the value is between 0 and 3 (inclusive), then the Pokémon must defend itself instead.

- Selecting a Pokémon's Next Attack or Defense Skill: When selecting the next offensive or defensive skill for a Pokémon, your system must generate an appropriately sized random integer value based on the number of possible options, and then select the skill that corresponds to that index/position in the list, and the list of skills must be sorted in ascending order based on the damage level for each skill.

  Suppose that a Pokémon has four attacks: (Aeroblast, damage: 100), (Fury Attack, damage: 15), (Meteor Assault, damage: 150), and (Shadow Punch, damage: 60). If the random value selected is 2, then the attack that is selected must be Aeroblast. They would 1st be ordered by damage to represent [Fury Attack, Shadow Punch, Aeroblast, Meteor Assault], then those could map to a random array of [0,1,2,3]; having random generate the 2, would be the 3rd index into the array, thus producing Aeroblast.

And the **`removeseed()`** command must nullify (i.e., remove) the existing seed values from the Pokémon. Your system must support a sequence of **`setseed()`** → **`removeseed()`** → **`setseed()`** commands before the battle commences, and the latest seed value must be used correctly.

Finally, the **`stop`** command should cause a graceful exit of the command input loop.

Displaying a Pokémon's Status

Your system must implement the capability to display the status of a Pokémon. This status display includes the name and current hit points of the Pokémon. It also includes the attacking and defensive skills, displayed separately in the ascending order of the damage levels. Here is an example of Charmander's status:

```
displayinfo,Charmander
> displayinfo,Charmander
Pokemon: Charmander has 25 hp
Attack Skills:
Name: Attack Damage: 1
Name: Scratch Damage: 2
Name: Ember Damage: 3
Name: Flamethrower Damage: 6
Defense Skills:
Name: Endure Defense: 1
Name: Block Defense: 2
Name: Protect Defense: 3
```

Conducting a Pokémon Tournament

Your system must be able to conduct a Pokémon Tournament. The tournament must be conducted as a set of knockout rounds until only one Pokémon remains victorious. Also, there are no Pokémon teams in this phase – each Pokémon is battling to win the tournament for itself.

The Pokémon will be listed in a specific order, and that order will determine how they must be paired for the tournament. You must verify that there are at least four (4) or more Pokémon participating in the tournament before the tournament begins.

Once verified you can start the tournament. As an example, in a tournament with four Pokémon, the 1st and 2nd Pokémon (as listed in order) must battle in the 1st round. In the (next) 2nd round, the 3rd and 4th Pokémon must battle. Finally, the winners of the 1st and 2nd rounds must battle in the 3rd and final round to determine the overall winner of the tournament.

[2] Implementing a Graphical User Interface (GUI)

Your application must provide a reasonable Graphical User Interface to support your application. An extremely complex interface is not required and is not the intent for this course. A simple interface that provides the required functionality will be sufficient.

- Even though you are implementing a GUI, you are still permitted to have Command-Line Interface (CLI) capabilities with your application to support development, maintenance, troubleshooting and execution functionality. Your GUI, however, must be able to support all the functionality required for a user to manage – and view the state of – a simulation run. Your CLI is allowed to provide "behind the scenes" support for the GUI as needed but should not be required for use during your TA's testing.
    - There will be sample code released showing how to connect a web GUI to a Java Docker container.
- We are giving your team wide latitude in selecting a framework to support the development of your user interface, given that you are responsible for installing and configuring that framework on the course-approved Docker Container (or an alternate pre-approved platform) for submission.
- Your user interface must support the requirements previously described, in addition to any updates described above. This is critical to our ability to evaluate your application thoroughly.

[3] Improved Error Checking & Handling

Your system must also handle the following types of errors appropriately. Demonstrations of all the appropriate responses will be provided in the test case expected results files.

1. *Invalid Numbers of Pokémon:* A valid number of Pokémon must be provided before a battle or tournament is allowed to begin/proceed.
2. *Invalid Pokémon Names:* All Pokémon names (i.e., references) must be validated before a battle is allowed to begin/proceed. If it is deemed that a Pokémon name is invalid, then that (invalid) Pokémon will immediately forfeit its match.

Sample Code

- We've included some sample Java source "starter code" that provides the command loop for terminal-based data input. We've also provided some test cases, though you are also welcomed (and highly encouraged) to develop your own tests as well.
- You are permitted to share your test cases with fellow students, but you are not permitted to share code and/or specific discussions or directions on the code or designs need to pass the cases.
- The links to the GitHub repositories will be shared on ed discussions.
    - 1 link will be focused on sample code for the Pokémon project
    - 1 link will be focused on how to add certain functional and non-functional aspects to a repository
    - You should investigate ways of combining these repositories and making them your own. Remember your project code should always remain private even after you finish the class or graduate from the program.

Closing Comments & Suggestions

This is the information provided by the customer so far. We (the OMSCS 6310 Team) will conduct Office Hours where you will be permitted to ask us questions to clarify the client's intent, etc. We will

answer some of the questions, but we will not necessarily answer all of them. Also, though this current version of the system is the "core" of the system going forward, our clients will add, update, and remove some of the requirements over the span of the course. One of your main tasks will be to ensure that your architectural documents and related artifacts remain consistent with the problem requirements – and with your system implementations – over time.

Quick Reminder on Collaborating with Others

Please use Ed Discussion for your questions and/or comments, and post publicly whenever it is appropriate. If your questions or comments contain information that specifically provides an answer for some part of the assignment, then please make your post private first, and we (the OMSCS 6310 Team) will review it and decide if it is suitable to be shared with the larger class. Best of luck to you with this assignment, and please contact us if you have questions or concerns.